

暗号理論

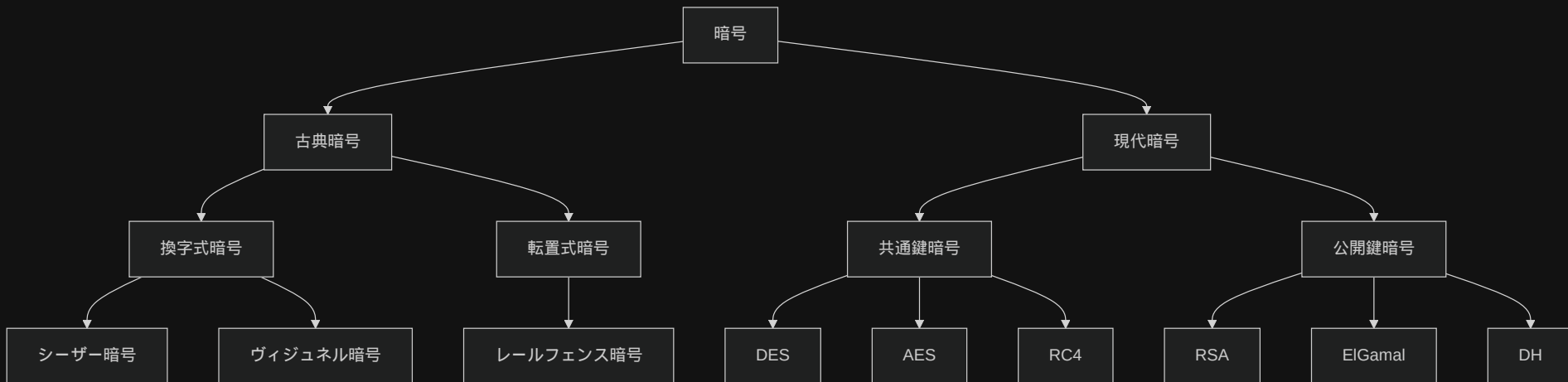
暗号理論の基礎を学ぼう

今日のゴール

- 暗号ってどういうのがあるんだっけ？を知る
- ツールを知る
- 簡単な数論を知る
- RSAについて理解する。

暗号とは

暗号は第三者に情報を盗まれないようにする目的を達成するもの。暗号の多くは古典暗号と現代暗号の2種類に分けられる。



シーザー暗号

古典暗号において変字式暗号がある。ここではシーザー暗号を紹介する。

シーザー暗号では平文に含まれるアルファベットをずらす事で暗号化する。複合の際はずれた文字を元に戻す事により平文に戻す。シーザー暗号ではアルファベットの三時のは案文である13をカギとしている。別名ROT13ともいわれる。

平文 A B C D ... X Y Z

↓ 鍵: 2

暗号文 C D E F ... Z A B

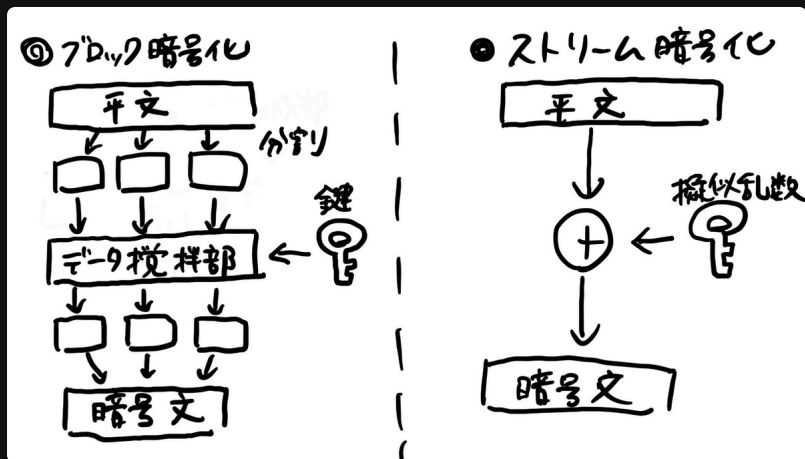
共通鍵暗号

共通鍵暗号は送信者と受信者が同じ鍵を用いて暗号化、複合化を行う暗号アルゴリズム。ブロック暗号とストリーム暗号に分かれる。

ブロック暗号：平文を一定長さのブロックに区切り、ブロック毎に暗号化を行っていく。ブロック暗号として有名なものはAESやDES。

ストリーム暗号：ブロック暗号よりも細かい単位で平文を区切り、逐次的な暗号化ができる方法。

鍵ストリームと平文を逐次XORする事によって、ブロック分のデータを準備せずともすぐさま暗号化が行える。



共通鍵暗号

ブロック暗号とストリーム暗号のメリデメは以下の通り。マルウェアでは通信、感染、リアルタイム処理にはストリーム暗号が使われ、ファイル暗号化はブロック暗号が使われる。

🔒 ブロック暗号 (Block Cipher)

メリット

- 高いセキュリティ性：構造がしっかりしており、解析耐性が高い
- 一貫性のある処理：ブロック単位で処理するため、整った暗号化が可能
- 多様なモード (CBC, ECB, CFB など) で柔軟な運用が可能

デメリット

- パディングが必要：ブロックサイズに満たないデータには補完が必要
- リアルタイム処理に不向き：一括処理のため、通信などの逐次処理には遅延が生じる

🌐 ストリーム暗号 (Stream Cipher)

メリット

- リアルタイム性が高い：受信した瞬間に暗号化・復号が可能
- 軽量で高速：処理がシンプルで、リソースの少ない環境でも使いやすい
- パディング不要：逐次処理のため、補完処理が不要

デメリット

- 安全性の検証が不十分なものもある：RC4などは脆弱性が指摘されている
- 鍵ストリームの管理が難しい：同期が必要で、漏洩すると全体が危険にさらされる

公開鍵暗号・デジタル署名

暗号化と復号に対してそれぞれ別のカギを用意し、暗号化用の鍵を公開する事から公開鍵暗号方式と言われている。公開する方の鍵を公開鍵、しないほうを秘密鍵という。公開鍵暗号として有名なものはRSA。

公開鍵暗号はビット数の大きい整数計算を多用するため、暗号化や復号に時間がかかるという欠点があり、CPUへの負荷も大きい。そこで、量が多いデータの通信には共通鍵暗号を使い、共通鍵だけを公開鍵暗号で送るハイブリット方式も良く使われる。また、公開鍵暗号はデジタル署名にも使われる。詳細は後で扱う。

暗号に対する攻撃の種類

5つ攻撃を紹介する

- 暗号文単独攻撃
 - 暗号文が入手できている状態から解読を試みる攻撃方法。
- 既知平文攻撃
 - ある暗号文に対する平文が入手できている状態から解読を試みる方法。
- 選択暗号文攻撃
 - 解読したい暗号文以外であれば任意の暗号文が復号できるような状況かで解読を試みる方法。
- 選択平文攻撃
 - 任意の平文を暗号化できる状態で解読を試みる方法。
- 関連鍵攻撃
 - 暗号化に使われた複数の鍵に数学的な関連性がある場合の解読を試みる方法。

環境構築

- PyCryptodome
- SageMath

PyCryptodome

PyCryptodomeは暗号系で使えるものがいっぱい入っているライブラリ。まずはインストール

```
pip install pycryptodome
```

非常に多くの機能が実装されているため詳細は公式ドキュメントを読んでみてください。

kaliではpipの制限がされているのでvenvでインストールするのが良いかもです

```
sudo apt install python3-venv  
python3 -m venv venv  
source venv/bin/activate  
pip install pycryptodome
```

PyCryptodome

pycryptodomeを使ってAESで暗号化してみる。

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad,unpad

# 暗号キー（共通鍵、秘密鍵）をバイト列で定義．暗号化の対象をmessageで定義．
key=b'16bytes_key12345'
message = b'helloworld'
# AES暗号器（Cipherオブジェクト）を生成．key を使ってAESを初期化．AES.MODE_ECB はECBモード
cipher=AES.new(key,AES.MODE_ECB)
# message を 16バイト境界までパディング．AESで暗号化．
enc=cipher.encrypt(pad(message,16))
# Cipherオブジェクトで暗号文を復号．復号結果はまだパディング済み．unpad で元の長さに戻す
dec=unpad(cipher.decrypt(enc),16)

print(enc)
print(dec)
```

実行結果

```
(venv)-(kali@kali)-[~/Desktop/pentest/01_study/crypt]
$ python cryptodome_AES.py
b'./x84\xa3h\x877\x83tM\xd0\xf2\x1d5\xa3\xc1L'
b'helloworld'
```

SageMath

SageMathは数学処理アプリケーション。二次方程式の解とか素因数分解とかできる。 kaliでaptですんなりとインストールできないので、docker上で起動しましょう。

```
sudo apt install docker.io  
sudo docker run -it sagemath/sagemath:latest
```

```
sudo apt install sagemath
```

数論

RSAを学びたいが、その前に数論を知る必要がある！！このあたりおさえればとりあえず大丈夫

- 合同式
- 逆元

合同式 ($\equiv$) とは？

暗号でよく使う「余りの世界」の数学
 $a \equiv b \pmod{m}$ の意味を直感的に学ぶ

合同式の基本

$a \equiv b \pmod{m}$ の読み方は？

a と b は m で割ったときの余りが等しい

例

- $17 \equiv 2 \pmod{5}$
($17 \div 5 = 3$ 余り 2 、 $2 \div 5 = 0$ 余り 2)

合同式は「余りの一致」

つまり...

- $12 \equiv 2 \pmod{10}$
- $22 \equiv 2 \pmod{10}$
- $32 \equiv 2 \pmod{10}$

どれも 10で割ると余りが2 → 同じ仲間

→ これを「合同クラス」と呼ぶ

合同式が暗号で重要な理由

巨大な数の計算を小さい世界に閉じ込められる

例：RSA

$$c = m^e \mod N$$

- m^e は数十万桁になる
- でも $\mod N$ を使えば $0 \sim (N-1)$ に収まる
- 計算・保存が現実的になる

合同式の性質（暗号に使うポイント）

計算してから余り

= 途中で余りを取っても結果は同じ

$$(a \bmod m)(b \bmod m) \bmod m = ab \bmod m$$

例

- $(12 \bmod 5) \times (9 \bmod 5) \bmod 5$

$$= (2 \times 4) \bmod 5$$

$$= 8 \bmod 5 = 3$$

- $12 \times 9 \bmod 5 = 108 \bmod 5 = 3$

→ 同じ！

→ これがRSAやDiffie-Hellmanを可能にする

合同式を理解する

円が目盛りを $0 \sim m-1$ とする

→ 時計のように値が「ループ」する世界

例 : mod 12

- $13 \rightarrow 1$
- $25 \rightarrow 1$
- $37 \rightarrow 1$

何周回っても同じ位置に戻る

→ 「余りの世界」が合同式

合同式の演算ルール

- 加法

$$a \equiv b \pmod{m} \Rightarrow a + c \equiv b + c \pmod{m}$$

- 乗法

$$ac \equiv bc \pmod{m}$$

- 冪乗

$$a^k \equiv b^k \pmod{m}$$

→ RSA の $m^e \bmod N$ はこの性質の上に成り立つ

総まとめ

- 合同式は「余りが等しい」という意味
- 巨大な数でも **mod** の世界で扱える
- 暗号 (RSA/DH) では必須の数学
- 計算途中で余りを取ってもOKな強い性質

逆元（ modular inverse ）とは？

暗号で超重要な概念

a の逆元とは「a と掛けて 1 になる数」

逆元の定義

a の逆元 a^{-1} は、次を満たす数：

$$a \cdot a^{-1} \equiv 1 \pmod{m}$$

例

- 3 の逆元 (mod 7) は 5
 - $3 \times 5 = 15$
 - $15 \bmod 7 = 1$

→ つまり「割り算」を mod の世界で実現する方法

逆元が存在する条件

a と m が「互いに素」なら必ず逆元が存在する

$$\gcd(a, m) = 1$$

例

- 3 と 7 は互いに素 → 逆元あり
- 2 と 4 は互いに素でない → 逆元なし

→ RSA の秘密鍵 d が必ず求まるのは
 e と $\phi(N)$ を互いに素にしているから

具体例（計算しながら理解）

問題：

7 の逆元 (mod 26) を求めよ

$$7x \equiv 1 \pmod{26}$$

いろいろ試すと

- $7 \times 15 = 105$
- $105 \bmod 26 = 1$

→ 答え：15

逆元を効率よく求める方法

拡張ユークリッドの互除法

最大公約数だけでなく

逆元 (modular inverse) を求めるためのアルゴリズム

普通のユークリッド法のおさらい

2つの整数 a, b に対して

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

つまり

余りに置き換えていくと、最終的に $\gcd$ (最大公約数) が出てくる。

拡張ユークリッド法とは？

目的は次の式を満たす x, y を求めること：

$$ax + by = \gcd(a, b)$$

特に $\gcd(a, b) = 1$ の場合、
 x が a の逆元 (mod b) になる。

逆元を求めたいとき

逆元の定義は：

$$a \cdot x \equiv 1 \pmod{m}$$

即ち以下に変換できる。

$$ax + my = 1$$

と書ける。

→ この x を探るのが拡張ユークリッド法。

例題：7 の逆元 (mod 26)

求めたいもの：

$$7x \equiv 1 \pmod{26}$$

つまり

$$7x + 26y = 1$$

「合同式」から「方程式」への変換

なぜ

$$7x \equiv 1 \pmod{26}$$

が

$$7x + 26y = 1$$

になるのか？

合同式の定義を思い出す

合同式

$$a \equiv b \pmod{m}$$

の意味は

a と b の差が m の倍数である

数学的には：

$$a - b = km \quad (k \in \mathbb{Z})$$

定義をそのまま使う

今回の式：

$$7x \equiv 1 \pmod{26}$$

合同式の定義より：

$$7x - 1 = 26k$$

これが出発点。

形を整える（1 を右辺に残す）

$$7x - 1 = 26k$$

両辺に 1 を足す：

$$7x = 26k + 1$$

整数方程式の形にする

$$7x = 26k + 1$$

両辺を 1 の形に揃える：

$$7x - 26k = 1$$

k の符号を変えて $y = -k$ とおくと：

$$7x + 26y = 1$$

これで変換完了

つまり

$$7x \equiv 1 \pmod{26}$$

は合同式の定義から直接

$$7x + 26y = 1$$

という整数方程式に変換できる。

Step 1 : 普通のユークリッド法を実行

$$26 = 7 \times 3 + 5$$

$$7 = 5 \times 1 + 2$$

$$5 = 2 \times 2 + 1$$

$$2 = 1 \times 2 + 0$$

gcd は 1 → 逆元は存在。

Step 2 : 式を逆向きに帰していく

まず :

$$1 = 5 - 2 \times 2$$

次に $2 = 7 - 5 \times 1$ を代入 :

$$1 = 5 - 2(7 - 5) = 3 \cdot 5 - 2 \cdot 7$$

Step 3 : さらに 5 を置き換える

$5 = 26 - 7 \times 3$ を代入 :

$$1 = 3(26 - 7 \cdot 3) - 2 \cdot 7$$

整理すると :

$$1 = 3 \cdot 26 - 11 \cdot 7$$

Step 4 : 結果を読む

係数比較 :

$$1 = 3 \cdot 26 + (-11) \cdot 7$$

$$\rightarrow x = -11$$

mod 26 で正の値にすると :

$$-11 \equiv 15 \pmod{26}$$

逆元が求まった

7 の逆元 (mod 26) は 15

チェック：

$$7 \times 15 = 105 \equiv 1 \pmod{26}$$

拡張ユークリッド法まとめ

- $\gcd$ を $ax + by$ の形で表す
- その x が逆元
- RSA の鍵生成（秘密鍵 d 計算）で必須
- 総当たりより圧倒的に高速

逆元が暗号で重要な理由

- RSA の秘密鍵 d は
 $ed \equiv 1 \pmod{\varphi(N)}$ を満たす逆元
- ECDSA / Diffie-Hellman でも大量に使う
- 「割り算」を可能にして
 - 鍵の生成
 - 復号
 - 署名の計算
が成立

逆元のまとめ

- 逆元とは「掛けて 1 になる数」
- 互いに素なら必ず存在
- 計算は拡張ユークリッド
- RSAを含むほとんどの公開鍵暗号で必須

RSA

RSA学んでいくよ

鍵生成

RSAでは

- 公開鍵 (Public key)
- 秘密鍵 (Secret key)

の2種類の鍵を生成する

鍵生成手順 (1)

1. 大きな素数 p, q を用意
2. $N = p \times q$ を計算
3. $\varphi(N) = (p-1)(q-1)$ を計算

鍵生成手順 (2)

4. $\varphi(N)$ と互いに素な整数 e を選ぶ

- 一般に $e = 65537$ を使用
- 小さくて計算しやすく、攻撃にも強い値

5. $ed \equiv 1 \pmod{\varphi(N)}$ を満たす d を計算

- これが秘密鍵となる

鍵の構造

- 公開鍵 : (N, e)
- 秘密鍵 : d

RSA の安全性

- N を素因数分解しない限り d は求まらない
- 素因数分解には膨大な時間が必要
- 現在のコンピュータでは 1024bit 以上が推奨

暗号化

RSA暗号の暗号化は非常にシンプル

- 平文： m
- 暗号文： c
- 公開鍵 (N, e) を使用

$$c \equiv m^e \pmod{N}$$

復号

暗号文 c を平文 m に戻すとき
秘密鍵 d を使って次の式で復号できる：

$$m \equiv c^d \pmod{N}$$

なぜ復号できるのか？

$c \equiv m^e$ なので $m^{ed} \equiv m \pmod{N}$ を証明すればいい

鍵生成より：

- $ed \equiv 1 \pmod{\phi(N)}$
- よって $ed = k\phi(N) + 1$

証明の流れ

$$m^{ed} = m^{k\varphi(N)+1} \pmod{N}$$

↓

$$= m^{k\varphi(N)} \cdot m$$

↓

$$m^{ed} \equiv (m^{\varphi(N)})^k \cdot m$$

証明の流れ

オイラーの定理から

$$m^{\varphi(N)} \equiv 1 \pmod{N}$$

なので：

$$m^{ed} \equiv m \pmod{N}$$

pemファイルからNとeを抽出

よく見るこんな感じの公開鍵からNとeを抽出する。これのどこにNとeがある？

```
└─(venv)-(kali㉿kali)-[~/Desktop/pentest/01_study/crypt]
└─$ cat public.pem
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAgk7lFUY/xwEcYLeC/uZ8
ni0m4IU5dI/sMPRD4kivgeQIK1UqPMUm7dkTNJAsgYCpmHGfNPQ+acrdR5BzeK9x
/Fmqy1S6vwqShnSJrEjW0dwJEjbnf6c34hMjYFL3c1zIkgTfaRoAgwdWv6iesY0h
/eLDtQH03FXRJ/J9/+zD60be6+ASUsoUYDLsuXltjxjEN/ICZGo3P/rj6sLg9GcE
PQfNHyH0SghXesTd15zLVn3zE0gHYiEoVw6z1Il0q0fe5airGnFbB4jYoJl/vt2g
apGXaU6bFB0PBZg0CDtemIQyQaMYarYUZEaz5AfGd3zVt46aoSqDjgs/kGb0eDx+
aQIDAQAB
-----END PUBLIC KEY-----
```


pemファイルからNとeを抽出

以下のpythonを実行

```
from Crypto.PublicKey import RSA
key = RSA.import_key(open("public.pem").read())
print("N =", key.n)
print("e =", key.e)
```

実行結果

```
└─(venv)-(kali㉿kali)-[~/Desktop/pentest/01_study/crypt]
└─$ python ne.py
N = 21499416347651022649474918422347555850500127571735842247904104640552636098728239010588407229902900278833170247626107268
e = 65537
```

RSA暗号に対する典型的な攻撃

RSAは普及率が高く、構造もシンプル

→ 不適切に使うと攻撃される

e や m が N に対して小さい場合

RSA暗号は本来：

$$c = m^e \bmod N$$

しかし m と N が近い or e が小さいと
暗号化しても安全でないことがある

条件

- e が小さい (例 : $e = 3$)
- m が十分小さい
- つまり :

$$m^e < N$$

このとき実は :

$$c = m^e$$

となり $\text{mod } N$ の意味がなくなる

つまり...

割り算していないので
暗号文は べき乗そのまま

→ 復号は `root` (べき根) を取るだけ

例：e = 3 で m が小さい RSA の復号

```
sage -q
p = random_prime(2^64); p
q = random_prime(2^64); q
N = p*q
e = 3
m = 1234
c = pow(m, e, N); c
int(c)^(1/3)
```

